Introduction

A static figure is what a journal prints; an interactive figure is what a colleague needs when exploring a result on a screen. `plotly::ggplotly()` converts an existing `ggplot` object into an HTML widget that supports hover tooltips, drag-to-zoom, panning, and selection brushing — without requiring any JavaScript. The result fits naturally into Quarto and RMarkdown reports, Shiny dashboards, and standalone HTML pages, and gives reviewers and collaborators a way to interrogate the data without exporting CSVs.

Prerequisites

A working knowledge of `ggplot2` (geoms, aesthetics, themes) and an output context that supports HTML rendering — Quarto, RMarkdown, Shiny, or RStudio's Viewer pane.

Theory

`ggplotly()` walks the structure of a `ggplot` object — its layers, scales, and aesthetic mappings — and translates each component into the corresponding `plotly.js` primitives. Hover tooltips default to whatever aesthetic mappings are in scope; custom tooltips come from mapping `aes(text = ...)` and passing `tooltip = "text"` to `ggplotly()`. The translation is faithful for most common geoms (point, line, bar, ribbon, density), partial for some statistical layers (`geom_smooth`, faceting with free scales), and weakest for non-standard extensions where you may need to drop down to native `plot_ly()` syntax.

Assumptions

The output environment renders HTML. Static targets (PDF, PNG, EPS) lose all interactivity, so for journal submission you should retain the original `ggplot` and re-render it with `ggsave()`; `plotly` is for screens, not paper.

R Implementation

```r
library(ggplot2); library(plotly)

p <- ggplot(mtcars, aes(wt, mpg, colour = factor(cyl),
                        text = rownames(mtcars))) +
  geom_point(size = 3) +
  theme_minimal()

ggplotly(p, tooltip = "text")

# Link two plots
p2 <- ggplot(mtcars, aes(hp, mpg)) + geom_point()
subplot(ggplotly(p), ggplotly(p2))
```

Output & Results

An interactive HTML scatter where each marker shows the car name on hover, the legend is clickable to toggle cylinder groups, and drag-zoom narrows the axes. `subplot()` arranges multiple `ggplotly` objects side by side with synchronised axes — useful for cross-filtering between related views.

Interpretation

A reporting sentence: "Figure 2 is an interactive scatterplot rendered with `plotly::ggplotly()`; hovering over a marker shows the car name and cylinder count, and clicking a legend entry toggles the corresponding group." Mention the technology in the figure caption when interactivity carries part of the figure's meaning so static fallbacks (a screenshot for the print version) are not mistaken for the whole story.

Practical Tips

- Render the static `ggplot` and the `ggplotly` version from the same source; that way the printed and online versions stay in sync.
- For custom tooltips, include only the variables that matter: too many columns under the cursor become a wall of text. `aes(text = paste("Name:", rownames(data), "<br>HP:", hp))` is a common pattern.
- `subplot()` combines multiple `ggplotly` objects with optional shared axes; for many panels, build them via `lapply()` and combine with `do.call(subplot, ...)`.
- Performance degrades beyond a few thousand points; switch to `plot_ly(..., type = "scattergl")` for WebGL rendering when working with tens of thousands.
- Inside Shiny, swap `plotOutput()` / `renderPlot()` for `plotlyOutput()` / `renderPlotly()` and the rest of the app remains unchanged.
- Use `config(displayModeBar = FALSE)` to suppress the toolbar for embedded reports where the toolbar is more clutter than feature.

R Packages Used

`plotly` for `ggplotly()`, `plot_ly()`, `subplot()`, and the JavaScript bindings; `ggplot2` for the static plot underlying the conversion; `htmlwidgets` for embedding the result in standalone HTML pages.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- ggplot2 grammar and multi-panel layouts